

Real-world (make it feel alive with streaming) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and what each chunk carries](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and what each chunk carries

Basic Version

Add one argument to the same call you already know: `stream=True`. Instead of getting one finished answer back, you get back something you loop over — a series of small pieces of the answer, one at a time, in order.

Use `print(piece, end="")` instead of a normal `print(piece)` — the default `print()` adds a newline after every call, which would put each little piece of text on its own line instead of joining them into one flowing sentence.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

With `stream=True`, `client.chat.completions.create(...)` returns an iterable of "chunk" objects instead of one finished response object. Each chunk carries a small piece of the answer at `chunk.choices[0].delta.content` — and this can be `None` (for chunks that don't carry new text, like the very first one), so you need to guard against printing `None`.

`print(text, end="")` — the `end=""` argument replaces the default trailing newline with nothing, so consecutive `print()` calls flow together on the same line, which is what makes streamed output look like it's "typing." Also handle the `None` case cleanly: `piece = chunk.choices[0].delta.content` can be `None`, so check `if piece:` before printing, or use `piece or ""` so you're never trying to print `None` itself.

If you also want to build up the full reply text (useful if you need to save it into conversation history afterward, like in the previous exercise), start with an empty string before the loop and add each piece to it as it arrives.

Difference between Basic and Intermediate: Basic and Intermediate both stream correctly for the ordinary, uninterrupted case — Intermediate explains precisely why `end=""` and the `None` guard are needed, and accumulates the full text for you to reuse.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
call the model with stream=True
for each piece that comes back:
    get the text out of this piece
    if there is text: print it, with no newline after
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# streaming_practice.py
stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    stream=True,
)

for chunk in stream:
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece, end="")

print()
```

Expected output: the reply appears on screen word by word (or a few characters at a time) instead of all at once, then a blank line from the final bare `print()` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function stream_reply(history) -> str:
    stream = call the API with messages=history, stream=True
    full_reply = ""
    for each chunk:
        piece = chunk's delta content
        if piece exists:
            print it, no newline
            add it to full_reply
    print a final newline
    return full_reply
```

```
# streaming_practice.py
def stream_reply(history: list) -> str:
    stream = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
        stream=True,
    )

    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply = full_reply + piece

    print()
    return full_reply
```

`full_reply` accumulates the whole answer as it streams by, so once the loop finishes you have the complete text — useful if you then need to append it to `history` the way the previous exercise did. Write and run this version, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic prints the reply correctly for a stream that completes normally. Intermediate wraps the same loop in a named, typed function that also accumulates and returns the full text — the exact shape the Build Task's `stream_message()` needs.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)